

UNIVERSIDADE DE BRASÍLIA

ENGENHARIA DE SOFTWARE

COLECTURE: ARQUITETURA DO PRODUTO DE SOFTWARE

GRUPO HAMILTON

ANA CLARA TEIXEIRA GUIMARÃES

ARTHUR MIGUEL SILVA ROCHA

GABRIEL SAYD DE SANTANA LOMBAS MACEDO

GUILHERME RIBEIRO ALMEIDA

HENRIQUE CATHARINO NUNES DE MENDONÇA

JÔNATAS MACIEL DA SILVA ALVES

LETÍCIA DA SILVA PEREIRA

RICIERI ARTUR VASCONCELLOS VIEIRA

BRASÍLIA – DF

2026

Tabela - Integrantes do Grupo

Matricula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
222021951	Gabriel Sayd de Santana Lombas Macedo	Elaboração de tópico 2	12.5
221022613	Jônatas Maciel da Silva Alves	Revisão do documento	12
251005883	Ana Clara Teixeira Guimarães	Elaboração do tópico 2 e revisão das normas ABNT	12.5
251005865	Arthur Miguel Silva Rocha	Elaboração do tópico 1 e 2	13
251021312	Guilherme Ribeiro Almeida	Revisão do documento	12
251021822	Henrique Catharino Nunes de Mendonça	Elaboração do tópico 1 e revisão do documento	13
251040256	Letícia da Silva Pereira	Elaboração do tópico 2 e revisão do documento	12.5
252014410	Ricieri Artur Vasconcellos Vieira	Elaboração do tópico 2 e revisão do documento	12.5

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
07/10/2026	1.0	Versão inicial com o detalhamento arquitetural, escopo e bibliografias	Grupo Hamilton
13/10/2026	1.1	Reintroduzir a camada de serviços nos textos e nas figuras, e atualizar a meta de testabilidade	Ricieri

Sumário

1 Introdução 5

1.1 Propósito 5

1.2 Escopo 5

1.3 Glossário 5

2 Representação Arquitetural 6

2.1 Definições 6

2.2 Justifique sua escolha arquitetural 6

2.3 Detalhamento 7

2.4 Metas e restrições arquiteturais 9

2.4.1 Metas Arquiteturais 9

2.4.2 Restrições Arquiteturais 10

2.5 Visões 11

2.5.1 Visão de uso 11

2.5.2 Visão de organização lógica 13

2.5.3 Visão estrutural 14

2.5.4 Visão de Implantação 15

2.6 Restrições adicionais 16

Bibliografia 18

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema CoLecture, desenvolvido pelo Grupo Hamilton na disciplina Métodos de Desenvolvimento de Software (MDS), no segundo semestre de 2026. Seu objetivo é oferecer uma visão geral das decisões arquiteturais, tecnologias e restrições do projeto a desenvolvedores, testadores e demais interessados, servindo de referência para a implementação e a manutenção do sistema.

1.2 Escopo

O CoLecture é uma plataforma web de leitura social. Seu escopo inclui: cadastro e gerenciamento de usuários e livros; comunidades de leitura e fóruns de discussão; registro de progresso e sistema de ofensiva (dias consecutivos de leitura); metas individuais e coletivas; avaliações de livros; conquistas; e controle de conteúdo, com níveis de acesso distintos para leitores, moderadores e administradores.. O detalhamento completo encontra-se no documento CoLecture: Visão do Produto e do Projeto (GRUPO HAMILTON, 2026).

1.3 Glossário

Termo	**Definição**
Comunidade	Grupo de usuários que leem e discutem o mesmo livro ou tema. Substitui os termos "grupo", "sala" e "sala de leitura". *(o termo substituía a si mesmo)*
Leitor	Usuário autenticado comum.
Moderador	Usuário com permissão para remover conteúdo e gerenciar as comunidades que modera.

Administrador	Usuário distinto do leitor, que gerencia comunidades, usuários e livros
Ofensiva	Número de dias consecutivos em que o leitor registrou progresso de leitura.
MTV	Model-Template-View, variação do MVC adotada pelo Django.

2 Representação Arquitetural

2.1 Definições

O CoLecture adota o estilo monolítico em camadas, cliente-servidor, no qual o framework Django organiza a aplicação segundo o padrão MTV (Model-Template-View), baseado conceitualmente no MVC (Model-View-Controller). No Django, o roteador de URLs (URLconf) cumpre parte do papel do controller, e a View concentra o tratamento das requisições (DJANGO SOFTWARE FOUNDATION, [s.d.]).

Os models representam os componentes do domínio (como livros e usuários) e definem e persistem os dados. As views recebem as requisições dos clientes, articulam a lógica necessária para atendê-las e produzem uma resposta, de modo semelhante ao controller do MVC. Os templates estruturam a interface gráfica e apresentam os dados ao usuário.

2.2 Justificativa da escolha arquitetural

A arquitetura MTV (Model-Template-View) do Django, de base conceitual MVC, foi escolhida por quatro razões que se reforçam. A primeira é a experiência da equipe: os integrantes já conhecem Python e Django, o que reduz a curva de aprendizado e permite concentrar o esforço nas regras de negócio. A segunda é que o Django oferece, de forma nativa, recursos essenciais ao escopo definido na Seção 1.2, como autenticação, controle de permissões para leitores e moderadores, painel administrativo, proteção contra CSRF e um ORM que simplifica a persistência no SQLite. A terceira é a separação em camadas, que permite aos oito integrantes

trabalhar em paralelo na interface (templates), na lógica de requisição (views e forms) e nos dados e regras de negócio (models), reduzindo conflitos de integração. A quarta é a aderência às metas da Seção 2.4.1: a renderização no servidor e o uso do ORM atendem à meta de desempenho com baixa complexidade.

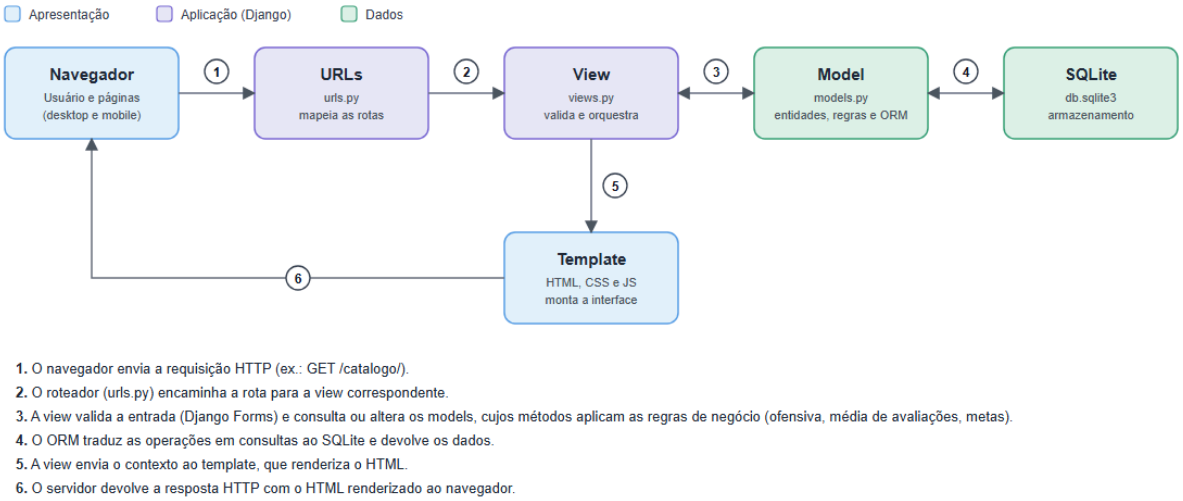
2.3 Detalhamento

A arquitetura do CoLecture é instanciada com o MTV anteriormente mencionado e com seu motivo justificado nos tópicos anteriores e será trabalhada da seguinte forma:

- **Model:** responsável pela estrutura de dados, pela persistência e pelas regras de negócio do domínio, expressas como métodos (como a média de avaliações do livro, a contagem de dias consecutivos da ofensiva e o progresso de uma meta). Representa as entidades do domínio (User, Profile, Book, Tag, Review, ShelfEntry, Community, Membership, Goal, Topic, Message, Achievement e Notification) e interage com o SQLite por meio do ORM do Django.
- **Template:** responsável pela interface gráfica (HTML, CSS e JavaScript), a camada que o usuário vê. Exemplos: página de login, catálogo de livros, página de leitura (com abas de progresso) e página da comunidade.
- **View:** recebe as requisições HTTP, valida a entrada (Django Forms), consulta ou altera os Models, que aplicam as regras de negócio, e renderiza o Template de resposta.

A Figura 1 apresenta o fluxo de uma requisição: o navegador envia uma requisição HTTP; o roteador de URLs a encaminha à View; a View consulta ou altera dados via Model; e, por fim, renderiza um Template que retorna HTML ao navegador.

Figura 1: Fluxo da arquitetura MTV.



Fonte: Elaborado pelos autores (2026)

A tabela a seguir mostra os elementos do estilo arquitetural para o CoLecture, com suas responsabilidades, interfaces e regras de uso.

Elemento	Responsabilidade	Entrada → Saída	Regras de uso
Navegador	Interface com o usuário; envia requisições e exibe HTML	Ações do usuário → requisição HTTP	Suporte a layout adaptativo.
URLs (urls.py)	Mapear rotas para views	URL e método HTTP → view	Toda rota exposta deve estar mapeada; rotas protegidas exigem login
View (views.py)	Validar a requisição, acionar os models e	HttpRequest → HttpResponse	Não escreve SQL diretamente (usa o ORM); mantém as

	escolher o template		regras de negócio nos métodos dos models; valida entradas com Django Forms
Model (models.py)	Definir entidades do domínio (Usuário, Livro, Comunidade, Meta, Progresso, Fórum), suas regras de negócio e a persistência	Objetos Python ↔ registros no SQLite	Alterações de esquema somente por migrations; regras de negócio como métodos testáveis sem requisição HTTP
Template	Apresentar dados em HTML, CSS e JavaScript (login, catálogo, leitura, comunidades)	Contexto da view → HTML	Sem regra de negócio; apenas lógica de exibição
Banco de dados (SQLite)	Armazenamento persistente	Consultas do ORM → dados	Acesso apenas via ORM

Os termos "Template (Visão)" e "View (Controle)" são analogias ao MVC e devem ser entendidos como aproximações.

2.4 Metas e restrições arquiteturais

2.4.1 Metas Arquiteturais

A fim de garantir que a solução consiga cumprir com os objetivos declarados no documento de visão de maneira satisfatória e eficiente, é necessário que seja construída com as metas arquiteturais a seguir especificadas:

- Desempenho: O servidor deve processar e enviar a resposta das páginas principais em até 0,8 segundo, sem considerar o tempo de recebimento pelo cliente, medido no ambiente local de desenvolvimento (JOSEPH, 2026).
- Testabilidade: as regras de negócio (como cálculo de ofensivas e de média de avaliações) devem poder ser testadas de forma unitária e isolada, conforme os objetivos do GQM do documento de visão. Por isso, essas regras ficam em métodos dos models e não nas views, o que permite testá-las sem uma requisição HTTP.
- Responsividade: a interface deve se adaptar a diferentes resoluções de tela (layout adaptável), garantindo a portabilidade prometida.
- Segurança: autenticação obrigatória para interação, senhas armazenadas com hash, proteção contra CSRF e XSS (recursos nativos do Django) e permissões por perfil.
- Manutenibilidade: cada módulo é um app Django independente, com baixo acoplamento e sem dependências circulares.

2.4.2 Restrições Arquiteturais

As restrições arquiteturais visam padronizar o desenvolvimento do projeto, de forma a melhor orientar os desenvolvedores em relação às boas práticas e facilitar o entendimento mútuo durante o trabalho em conjunto. São elas:

- O back-end deve ser desenvolvido em Python, em conjunto com o framework Django.
- O front-end será servido por templates do Django (HTML, CSS e JavaScript).
- As requisições devem ser analisadas e validadas antes do processamento (por exemplo, com Django Forms), verificando se o formato é adequado.
- O banco de dados inicial será o SQLite, pela simplicidade de configuração e por dispensar servidor de banco.
- Os commits devem seguir o padrão Conventional Commits.
- Os identificadores (variáveis, funções, classes e tabelas) devem ser escritos em inglês, e os comentários e a documentação em português. A padronização reduz o contraste de idiomas com as palavras reservadas do Python e facilita a compreensão imediata das regras de negócio.

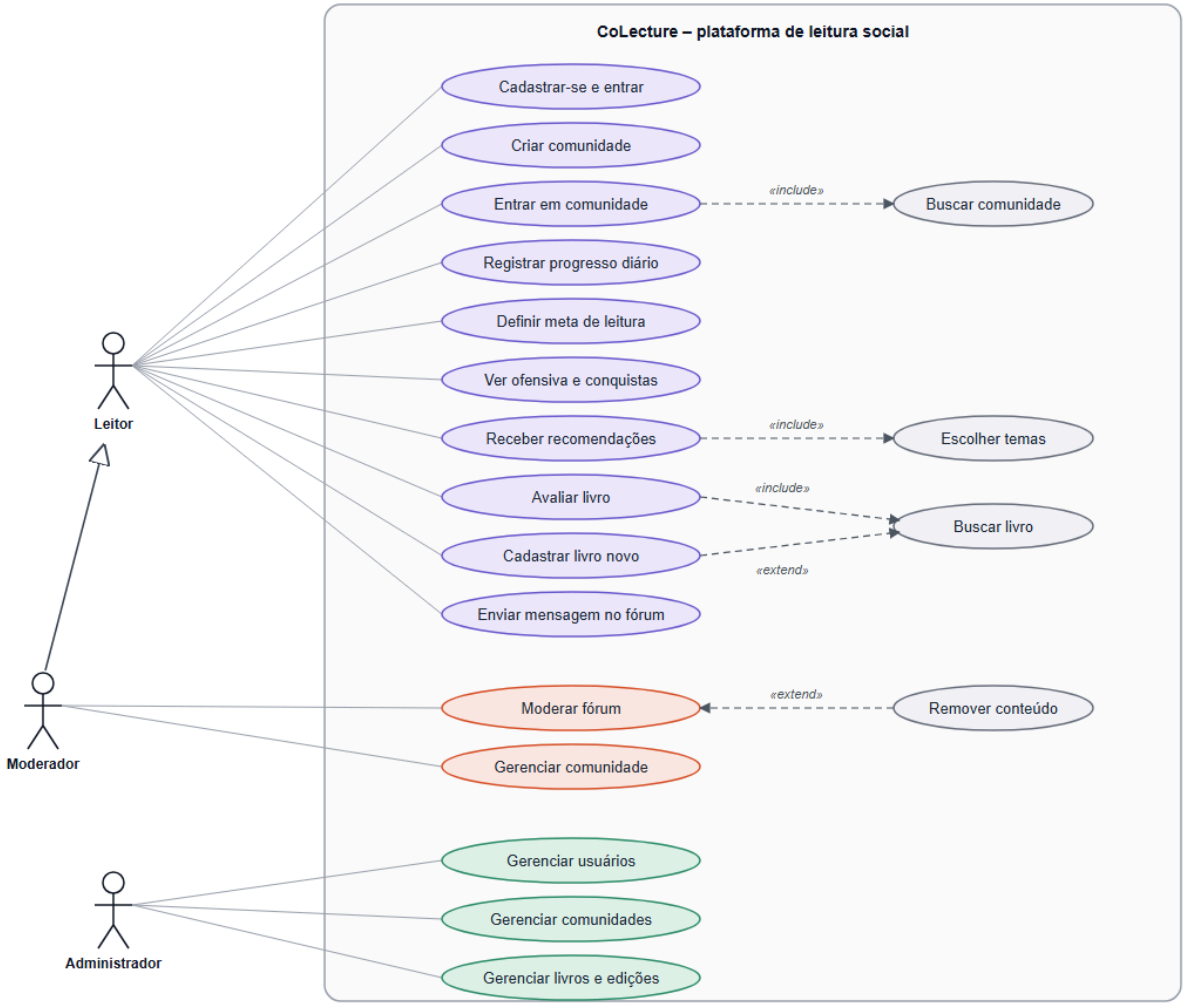
2.5 Visões

Esta seção apresenta a arquitetura segundo quatro visões: de uso, de organização lógica, estrutural e de implantação.

2.5.1 Visão de uso

O CoLecture permite que leitores criem e participem de comunidades de leitura, registrem seu progresso diário, interajam em fóruns, avaliem livros, cadastrem obras ainda ausentes do catálogo (livros recentes ou pouco conhecidos) e recebam recomendações baseadas em seus temas preferidos. Moderadores acompanham os fóruns e removem conteúdo inadequado. O diagrama de caso de uso da plataforma é apresentado na Figura 2.

Figura 2: Diagrama de caso de uso da plataforma.



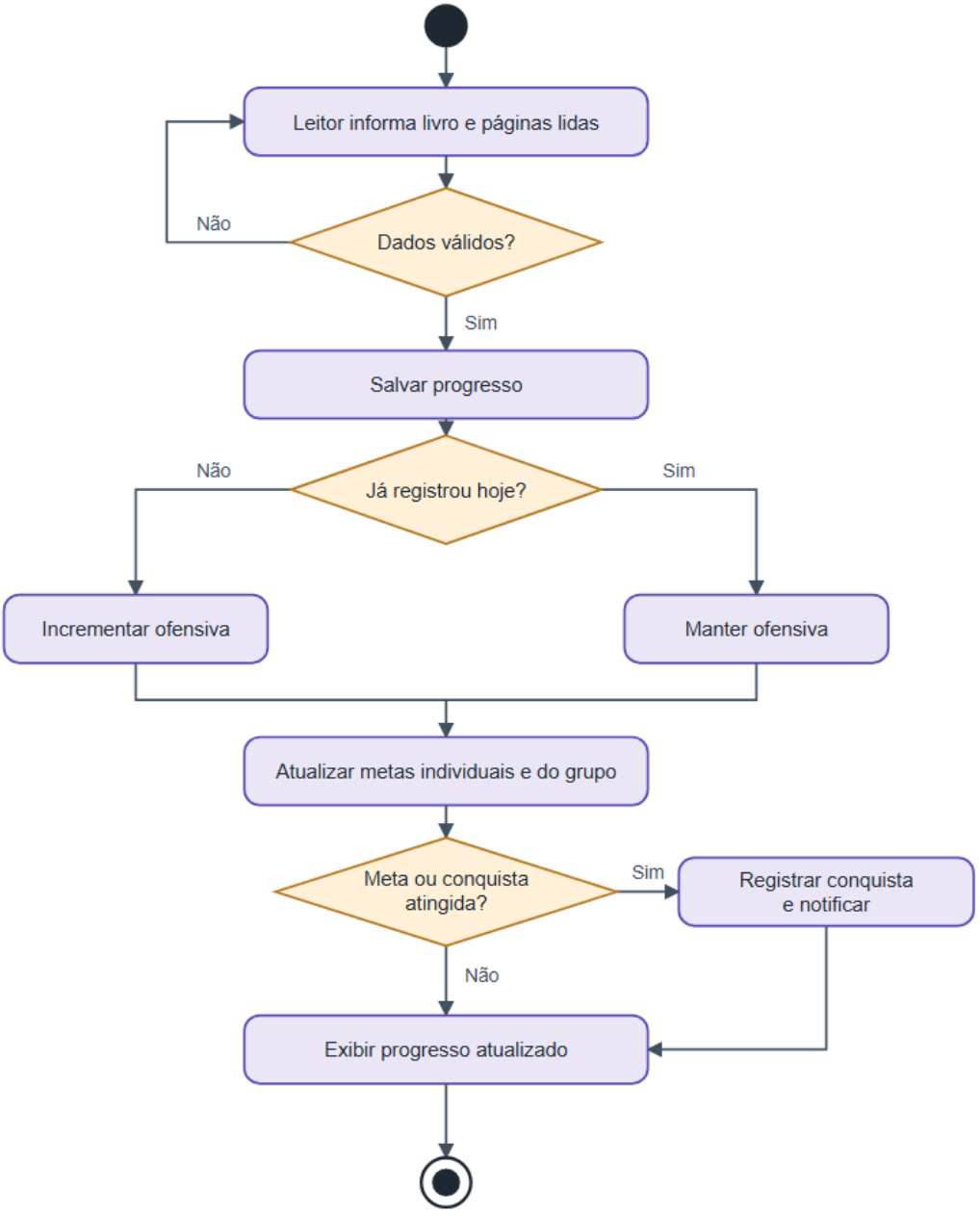
Fonte: Elaborado pelos autores (2026)

Três características do projeto influenciaram a escolha do MTV/Django: (i) o sistema é orientado a páginas e formulários (cadastro, avaliação, fórum), o que

combina com a renderização no servidor; (ii) há perfis distintos (leitor, moderador de comunidade e administrador), atendidos pela autenticação e pelas permissões nativas do Django e pelo papel de cada membro na comunidade; (iii) a equipe de oito integrantes já conhece Python e Django.

A funcionalidade mais complexa é o registro do progresso diário, que envolve validação, atualização da ofensiva, das metas e das conquistas. O diagrama de atividades é apresentado na Figura 3 .

Figura 3: Diagrama de atividades

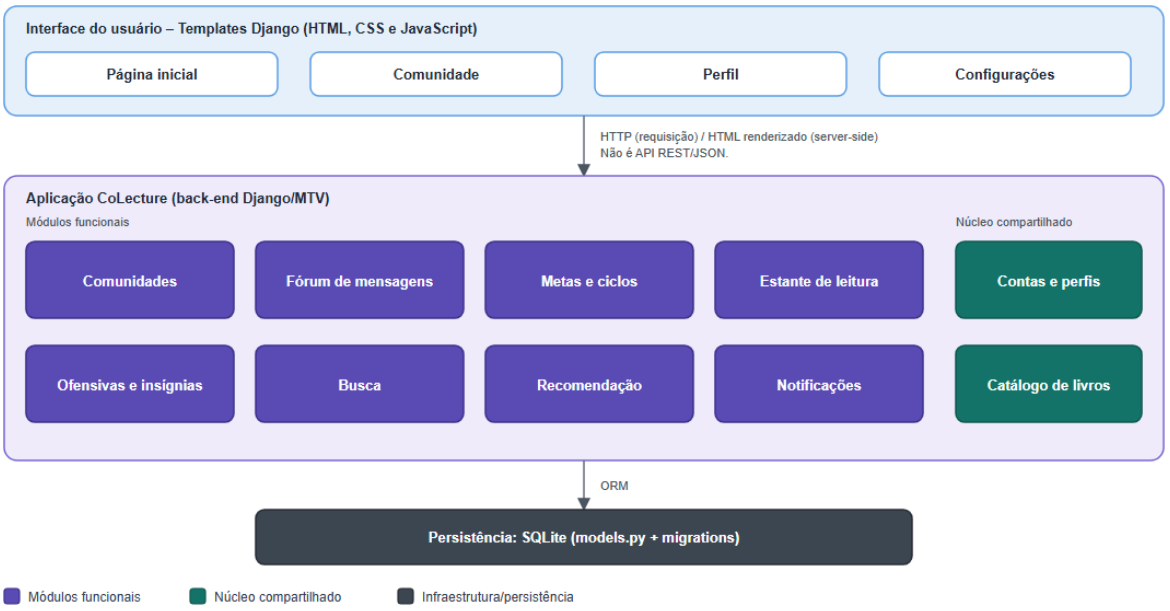


Fonte: Elaborado pelos autores (2026)

2.5.2 Visão de organização lógica

Ao acessar o aplicativo, o usuário será direcionado para a página inicial, onde será possível acessar as comunidades de leitura de que participa, usar um mecanismo de busca, para procurar comunidades já existentes no CoLecture, acessar a sua própria conta e as configurações do aplicativo, e criar uma nova comunidade ou entrar em uma já existente. O objetivo é oferecer fácil acesso às funcionalidades de comunidades e a configurações rápidas da conta. Ao entrar em uma comunidade (nova ou já existente), haverá um fórum de mensagens para interagir com outras pessoas e as informações da comunidade (nome, data de criação, moderadores, livro ou tema atual e participantes), de modo que as novas mensagens fiquem visíveis logo ao entrar e a troca de opiniões comece imediatamente. A busca cobre livros e comunidades públicas. Os status são só Quero ler, Lendo e Lido. O perfil não tem foto nem livro favorito. O próprio usuário poderá alterar essas informações em seu perfil. O propósito é permitir que os usuários encontrem pessoas com gostos parecidos e se aproximem por meio de comunidades de interesse em comum. A Figura 4 apresenta o diagrama dos módulos do sistema.

Figura 4: Módulos do sistema CoLecture(MTV/Django).

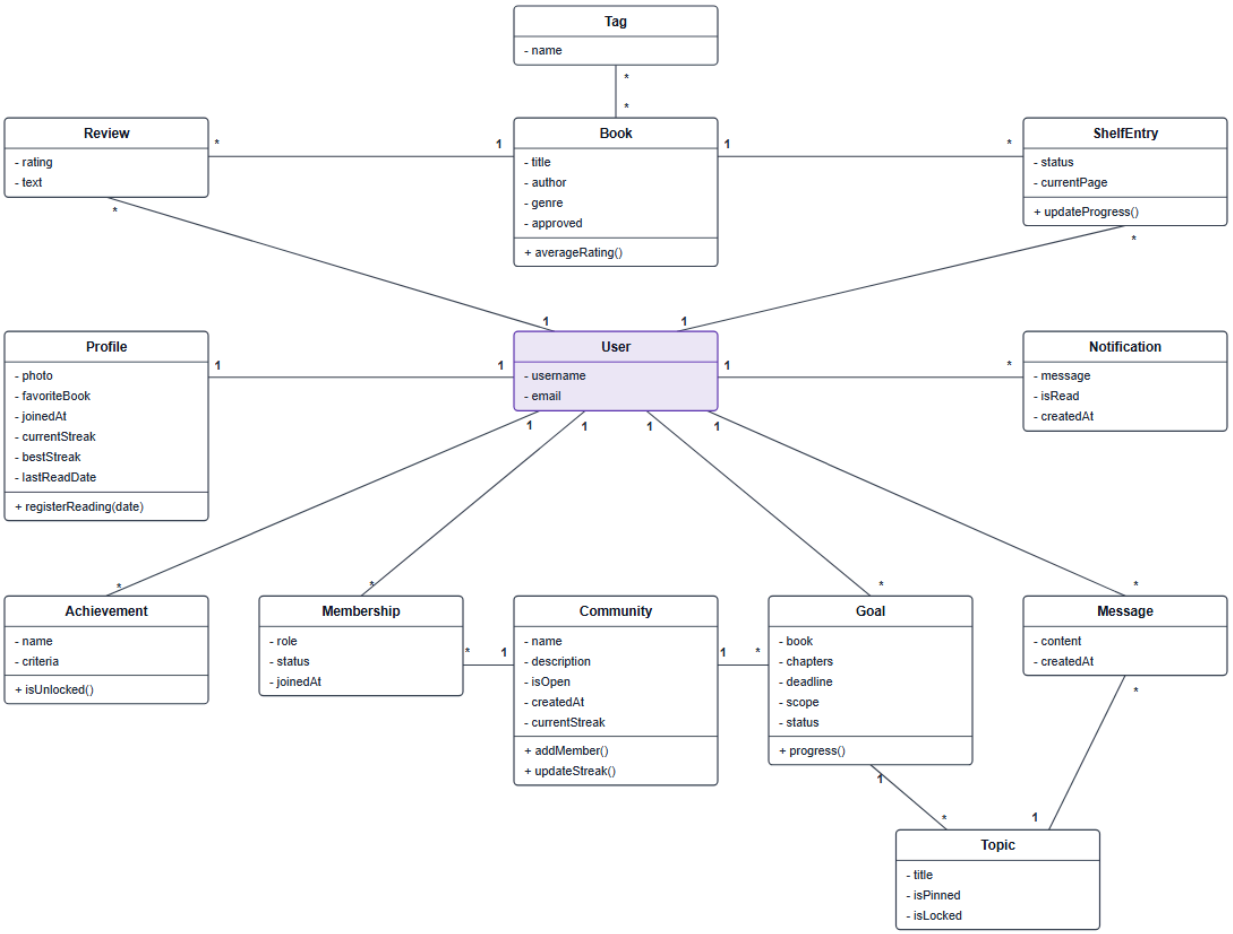


Fonte: Elaborado pelos autores (2026).

2.5.3 Visão estrutural

A estrutura do CoLecture segue a organização de um projeto Django: cada módulo da Seção 2.5.2 é um app independente, com seus próprios models, views, templates e testes. As principais entidades do domínio e suas relações estão na Figura 5. O usuário (User) concentra as relações, pois possui um perfil (Profile), que guarda a ofensiva pessoal e os dados de leitura, além de metas, conquistas, avaliações, mensagens, notificações, entradas na estante de leitura (ShelfEntry) e participações em comunidades. O livro (Book) recebe tags (Tag), é avaliado (Review) e registrado na estante. A comunidade (Community) reúne usuários por meio de Membership, cujo atributo role define se o membro é leitor ou moderador e cujo status indica se a entrada foi aprovada. A comunidade tem metas de leitura (Goal), e cada meta, ou ciclo, possui o seu fórum, formado por tópicos (Topic) que contêm mensagens (Message). Cada entidade corresponde a um model Django persistido no SQLite, com atributos privados acessados pelos métodos da própria classe, que concentram as regras de negócio, como pode ser visto na Figura 5.

Figura 5: Diagrama UML



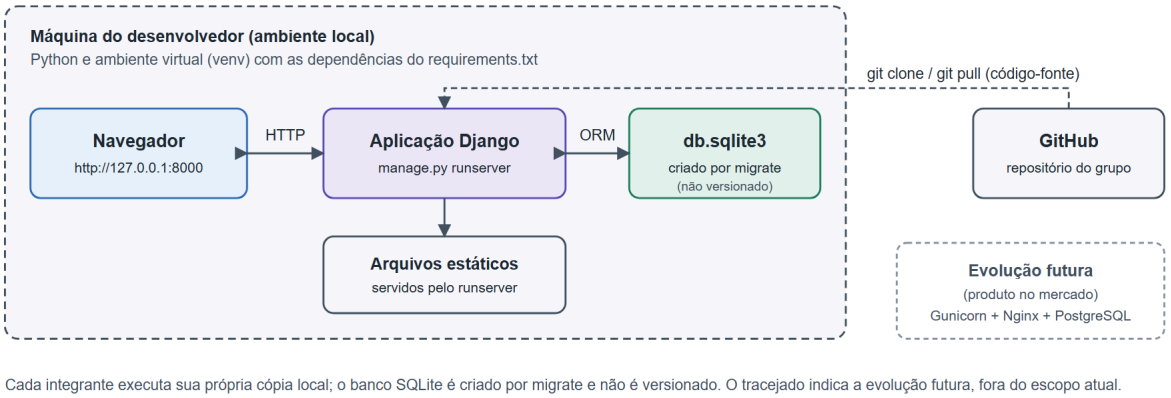
Fonte: Elaborado pelos autores (2026).

2.5.4 Visão de Implantação

O CoLecture será executado apenas em ambiente local, na máquina de cada integrante do grupo, sem hospedagem em provedor externo. O código-fonte é versionado no GitHub e obtido por git clone e git pull. Em cada máquina, a aplicação roda em um ambiente virtual Python (venv), com as dependências instaladas a partir do arquivo requirements.txt, e é iniciada pelo servidor de desenvolvimento do Django (python manage.py runserver). O banco de dados é um arquivo SQLite (db.sqlite3) criado localmente pelo comando migrate, não versionado, o que dispensa a instalação de um servidor de banco. Os arquivos estáticos também são servidos pelo servidor de desenvolvimento. O acesso se dá pelo navegador em http://127.0.0.1:8000. A Figura 6 apresenta o diagrama de implantação.

Essa escolha atende ao escopo da disciplina, pela simplicidade de configuração e pela independência de infraestrutura, e mantém a arquitetura compatível com uma futura implantação em servidor (Gunicorn, Nginx e PostgreSQL), caso o produto fosse levado ao mercado.

Figura 6: diagrama de implantação



Fonte: Elaborado pelos autores (2026).

2.6 Restrições adicionais

O CoLecture é uma aplicação web executada em ambiente local e acessada por navegador, exigindo autenticação para interagir em fóruns e comunidades.. As características de qualidade relevantes são:

- Segurança: autenticação obrigatória, senhas com hash, proteção contra CSRF e XSS e controle de acesso por perfil. Dados pessoais, como foto de perfil, serão tratados conforme a Lei Geral de Proteção de Dados (Lei nº 13.709/2018).
- Portabilidade e responsividade: interface com layout adaptável a diferentes tamanhos de tela.
- Usabilidade: interface simples, que permita o uso por pessoas com pouca experiência digital.

As permissões por perfil são:

Perfil	Permissões
Visitante	Visualizar páginas públicas, cadastrar-se e entrar
Leitor	Tudo do visitante, além de criar e entrar em comunidades, registrar progresso, avaliar livros, cadastrar livros e participar de fóruns
Moderador	Tudo do leitor, além de remover conteúdo e gerenciar membros das comunidades que modera
Administrador	Gerenciar comunidades, usuários e livros (editar e aprovar livros e edições) pelo painel administrativo.

Bibliografia

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 4. ed. Boston: Addison-Wesley, 2021.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

DJANGO SOFTWARE FOUNDATION. Django documentation. [S. l.], [s. d.]. Disponível em: https://docs.djangoproject.com/. Acesso em: 7 out. 2026.

GRUPO HAMILTON. CoLecture: Visão do Produto e do Projeto. Brasília, DF: Universidade de Brasília, 2026.

JOSEPH, Melwyn. What is a Good Website Response Time? (2026 Standards). 7 jul. 2026. Disponível em: https://www.webyes.com/blogs/good-website-response-time/. Acesso em: 6 out. 2026.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.